

Filesystem

Caminhos

Os caminhos (paths) são representações de localizações no sistema de arquivos. Eles podem ser absolutos ou relativos. Nesta biblioteca, a classe `std::filesystem::path` é usada para manipular caminhos de forma segura e eficiente.

Criação de um Caminho

Para criar um objeto `std::filesystem::path`, você pode simplesmente passar uma string representando o caminho:

```
#include <iostream>
#include <filesystem>
namespace fs = std::filesystem;

int main() {
    fs::path caminho_fonte = "src/main.cpp";
    std::cout << "Caminho: " << caminho_fonte << std::endl;
    return 0;
}
```

Neste exemplo, criamos um caminho relativo para o arquivo `main.cpp` localizado na pasta `src`. Podemos também pegar de volta o nome do arquivo (com e sem extensão), o diretório pai ou sua extensão:

```
caminho_fonte.filename(); // "main.cpp"
caminho_fonte.stem();     // "main"
caminho_fonte.parent_path(); // "src"
caminho_fonte.extension(); // ".cpp"
```

Concatenação

A classe `std::filesystem::path` oferece várias operações úteis para manipular caminhos. Você pode concatenar caminhos usando o operador `/` de forma segura, sem se preocupar com barras duplicadas:

```
fs::path diretorio = "src";
fs::path arquivo = "main.cpp";
fs::path caminho_completo = diretorio / arquivo;
```

ou

```
fs::path path = "src";
path /= "main.cpp";
```

Obs: use `fs::path(getenv("HOME"))` para pegar o diretório home do usuário (UNIX)

Navegação e Leitura

Iteração Simples

Você pode navegar pelo sistema de arquivos usando `fs::directory_iterator` para listar arquivos e diretórios apenas no nível atual:

```
for (auto parte : fs::directory_iterator(".")) {
    if (parte.is_directory())
        std::cout << "[DIR] " << parte.path().filename() << std::endl;
    else
        std::cout << "[ARQ] " << parte.path().filename() << std::endl;
}
```

Iteração Recursiva

Para entrar automaticamente em todas as subpastas, utilize o `recursive_directory_iterator`. Isso é ideal para varreduras profundas ou busca de arquivos em projetos complexos.

```
// Percorre pasta atual e todas as subpastas
for (auto parte : fs::recursive_directory_iterator(".")) {
    std::cout << parte.path() << std::endl;
}
```

Ações e Manipulação

Criar Diretórios

Use `create_directory` para criar uma única pasta, ou `create_directories` (plural) para criar toda a árvore de caminhos se ela não existir (similar ao `mkdir -p`).

```
fs::create_directory("projeto");
fs::create_directories("projeto/src/include"); // Cria tudo que falta
```

Links Simbólicos

Links simbólicos (atalhos) são essenciais para referenciar arquivos sem duplicá-los. **Dica: Use caminhos absolutos para evitar links quebrados.**

```
fs::path alvo = fs::absolute("projeto_alpha");
fs::path link = "armazem/atalho_alpha";
fs::create_directory_symlink(alvo, link);
```

Mover e Renomear

Em sistemas de arquivos, “mover” um arquivo para outra pasta ou apenas “renomear” o arquivo na mesma pasta é a mesma operação. A função `fs::rename` lida com ambos.

```
fs::path origem = "foto.jpg";
fs::path destino = "imagens/ferias.jpg";

// Requer que a pasta 'imagens' já exista!
fs::rename(origem, destino);
```

Copiar

Para copiar arquivos ou diretórios, use `fs::copy`. Existem opções (`copy_options`) para controlar se deve sobrescrever ou copiar recursivamente.

```
// Copia arquivo
fs::copy("origem.txt", "copia.txt");

// Copia diretório recursivamente
fs::copy("pasta_src", "pasta_dest", fs::copy_options::recursive);
```

Remove

`fs::remove("file.txt")`: Remove um arquivo ou um diretório vazio.

`fs::remove_all("pasta")`: Remove o diretório e todo o seu conteúdo (cuidado!).

Metadados e Informações

Além de manipular, podemos extrair informações vitais sobre os arquivos.

Tamanho e Tempo

```
if (fs::exists("arquivo.bin")) {
  // Tamanho em bytes
  uintmax_t tamanho = fs::file_size("arquivo.bin");

  // Data da última modificação
  auto tempo = fs::last_write_time("arquivo.bin");
}
```

Espaço em Disco

Útil para verificar armazenamento disponível antes de operações grandes.

```
fs::space_info info = fs::space("/");
std::cout << "Livre: " << info.free << " bytes" << std::endl;
std::cout << "Capacidade: " << info.capacity << " bytes" << std::endl;
```

Tratamento de Erros e Segurança

Por padrão, a biblioteca lança exceções (`std::filesystem::filesystem_error`) se algo der errado. Para aplicações robustas (como TUIs), prefira a versão que retorna códigos de erro (`std::error_code`) para evitar que o programa feche abruptamente.

```
std::error_code ec;
// Tenta criar pasta. Se falhar, não crasha, apenas preenche 'ec'
if (!fs::create_directory("/root/proibido", ec)) {
  std::cout << "Erro: " << ec.message() << std::endl;
}
```

Erros Comuns (Troubleshooting)

Aqui estão as armadilhas mais frequentes ao usar a biblioteca:

1. **“File exists” ao criar Link:** Tentar criar um link simbólico onde já existe um arquivo/link com o mesmo nome causará erro. Sempre verifique com `!fs::exists()` antes.
2. **Pai Ausente no Rename:** `fs::rename("a", "b/c")` falhará se a pasta `b` não existir. O `rename` move o arquivo, mas não cria a estrutura de diretórios do destino.
3. **Links Quebrados:** Criar symlinks usando caminhos relativos pode falhar se você mover o link depois. Prefira `fs::absolute()` ou `fs::canonical()` para o alvo do link.
4. **Iterador Inválido:** Não delete ou modifique a estrutura do diretório enquanto itera sobre ele com `directory_iterator`. Guarde os caminhos num `vector` e processe-os depois.